

Design and Implementation of an FPGA-Based Audio Filtering and FFT Analysis System

Course: FPGA Based System Design

Course Code: ELE 4411

Assessment: IA4

Project Report

Submitted by

S. No.	Name of Group Member	Roll No.
1	Ashwin Shankar	230906120
2	Nikita Nayak	230906046
3	Shadan Jawed	230906134
4	Rhea Sinha	230906116
5	Aditya Kini	230906436

School of Electrical Engineering
Manipal Institute of Technology, Manipal
April 22, 2026

Contents

1	Introduction	2
2	Design / Methodology	2
2.1	Stage-wise Block Diagram	3
2.2	Methodology Description	4
3	Results and Analysis	4
3.1	Timing Analysis	4
3.2	Power Analysis	5
3.3	FFT Plots Before and After Filtering	6
3.4	Device Utilization	7
4	Conclusion	9
5	Future Scope	9
6	References	9
7	Individual Contribution	10

1. Introduction

This report presents the implementation and analysis of an FPGA-based signal processing system designed as part of the course **ELE 4411: FPGA Based System Design**. The objective of this work is to develop a hardware-oriented design flow for digital signal processing tasks and evaluate the system in terms of performance, resource usage, and implementation feasibility.

The implemented design focuses on processing audio input data using FPGA-based computation stages. The design includes memory-based audio sample feeding, digital filtering, and Fast Fourier Transform (FFT) analysis. By implementing these stages on an FPGA platform, the design demonstrates the suitability of reconfigurable hardware for high-speed and parallel digital signal processing applications.

The project aims to study the complete flow from model design and HDL generation to synthesis, implementation, and performance analysis. The obtained results are used to compare the spectrum before and after filtering and to assess the timing, power, and hardware resource requirements of the design.

2. Design / Methodology

The overall design methodology followed in this work is stage-wise and consists of the following major blocks:

1. Audio Sample Preparation and Memory Initialization

In the first stage, the input audio signal is converted into a digital sample format suitable for FPGA processing. The audio samples are scaled and represented in signed fixed-point form so that they are compatible with the generated HDL design. These processed samples are then stored in a memory initialization file (`.mem`), which acts as the source of input data during simulation or hardware execution. This stage is important because it bridges the gap between the original audio waveform and the FPGA-compatible numerical representation required by the design.

2. Input Sample Feeding into the FPGA Design

In this stage, the stored audio samples are sequentially read from memory and fed into the FPGA design at the desired sample rate. A clock divider or sample strobe mechanism is used to ensure that the samples are applied to the processing chain at the correct timing interval. The validity signal associated with the input data ensures that only meaningful samples are processed by the downstream blocks.

This stage essentially emulates the behavior of a real-time data acquisition interface using preloaded digital samples.

3. FFT Computation of the Raw Audio

The raw input audio samples are first passed through an FFT block to obtain the frequency-domain representation of the original signal. The FFT transforms the time-domain samples into complex frequency bins, consisting of real and imaginary components. This allows the spectral content of the unfiltered input audio to be examined. The raw FFT output serves as a baseline for comparison with the filtered output spectrum in later stages.

4. FIR Filtering of the Input Audio

In parallel with the raw FFT path, the same audio samples are applied to a finite impulse response (FIR) filter. The purpose of this stage is to selectively modify the spectral content of the input signal by attenuating unwanted frequencies and preserving the desired band of operation. The FIR filter coefficients determine the filtering characteristics, and the output of this stage is a modified time-domain audio sequence. This stage demonstrates the FPGA implementation of digital filtering in a hardware-efficient manner.

5. FFT Computation of the Filtered Audio

The output of the FIR filter is then passed through a second FFT block to obtain the frequency-domain representation of the filtered signal. By comparing this FFT output with the FFT of the raw audio, the effect of filtering can be clearly observed. Frequency components removed or attenuated by the filter appear reduced in magnitude, while the preserved frequency components remain dominant. This stage enables direct spectral comparison before and after filtering.

6. Analysis of Implementation Results

In the final stage, the hardware implementation results are studied to evaluate the design from both functional and hardware perspectives. Functional verification includes comparison of the FFT plots before and after filtering. Hardware evaluation includes timing analysis, power analysis, and resource utilization such as logic cells, DSP slices, and BRAM usage. Design Rule Check (DRC) results are also examined to confirm that the implementation satisfies FPGA design constraints. This stage provides the basis for assessing the efficiency and feasibility of the complete FPGA-based system.

2.1 Stage-wise Block Diagram

Figure 1 shows the stage-wise block diagram of the implemented system.

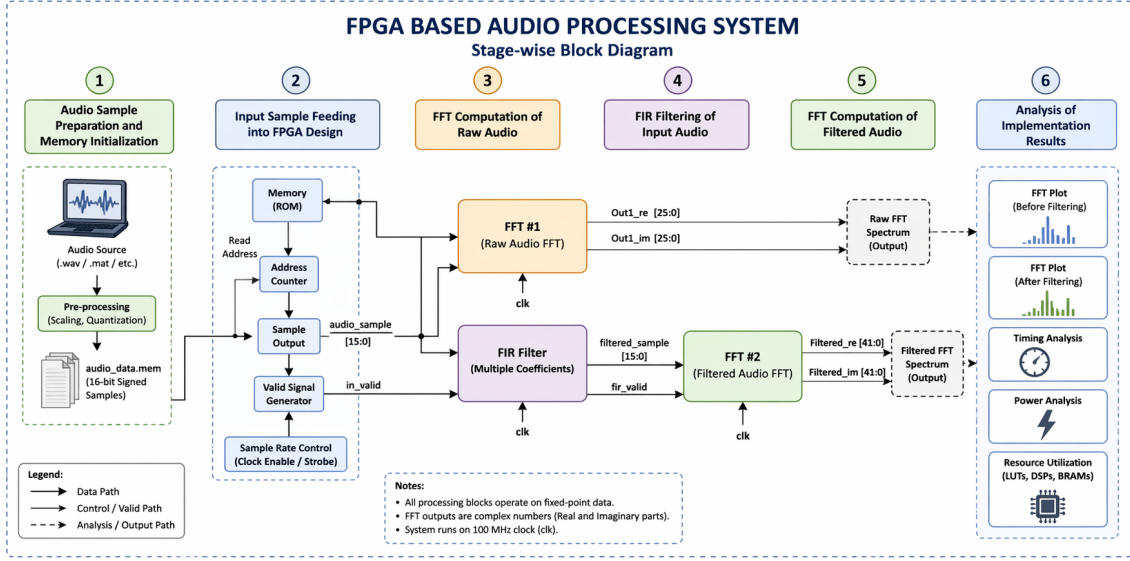


Figure 1: Stage-wise block diagram of the FPGA-based audio processing system

2.2 Methodology Description

The input audio signal is first converted into a suitable digital sample format and stored in a memory file. These samples are then fed into the FPGA design at the required sampling rate. The raw audio samples are passed to an FFT block to obtain the spectrum of the input signal.

In parallel, the same audio samples are processed by a digital FIR filter. The filtered output is then passed to another FFT block to observe the spectral characteristics after filtering. This enables a direct comparison between the raw and filtered signals in the frequency domain.

The complete design is synthesized and implemented in Vivado, and the generated reports are used for timing, power, and hardware resource analysis.

3. Results and Analysis

This section presents the implementation results and discusses the obtained outputs with respect to timing performance, power consumption, spectral characteristics, and resource utilization.

3.1 Timing Analysis

Timing analysis is an essential part of FPGA implementation, as it determines whether the design can operate correctly at the intended clock frequency. In this work, the timing performance of the implemented system was evaluated after synthesis and implementation

using the Vivado timing summary report. The main parameters considered were setup timing, hold timing, and the maximum achievable clock frequency.

Setup timing analysis verifies whether the data launched from a source register reaches the destination register within the required clock period. This is commonly evaluated using the Worst Negative Slack (WNS) and Total Negative Slack (TNS) values. If the setup slack is non-negative, the design is considered to satisfy setup timing. A negative setup slack indicates that the data path delay exceeds the allowable clock period, thereby violating the setup requirement.

Hold timing analysis ensures that the data remains stable for the required duration after the active clock edge. This is evaluated using the Worst Hold Slack (WHS) and Total Hold Slack (THS). If the hold slack is non-negative, the hold timing requirement is satisfied. A negative hold slack indicates a hold time violation and may lead to unreliable operation.

The maximum operating frequency of the design is determined from the critical path delay. For a worst-case path delay of **10.948 ns**, the corresponding maximum achievable clock frequency is:

$$f_{\max} = \frac{1}{10.948 \times 10^{-9}} \approx 91.34 \text{ MHz}$$

This indicates that the design can theoretically operate up to approximately **91.34 MHz** without setup timing violations. If the target clock frequency is higher than this value, the design may fail setup timing unless further optimization is carried out.

The timing analysis results provide an indication of the performance limits of the proposed FPGA-based system. If the setup and hold constraints are satisfied, the design is considered timing-safe for the specified operating conditions. Otherwise, additional optimization in terms of logic depth reduction, pipelining, or routing improvement may be required.

Table 1: Timing analysis summary

Parameter	Value
Setup Timing Status	Met
Hold Timing Status	Met
Maximum Clock Frequency	91.34 MHz

3.2 Power Analysis

The power analysis report provides an estimate of the total on-chip power consumption. The static and dynamic power components can be used to understand the energy requirement of the implemented design.

Table 2: Power analysis summary

Parameter	Value
Total On-Chip Power	129.793W
Dynamic Power	129.002W
Static Power	0.791W

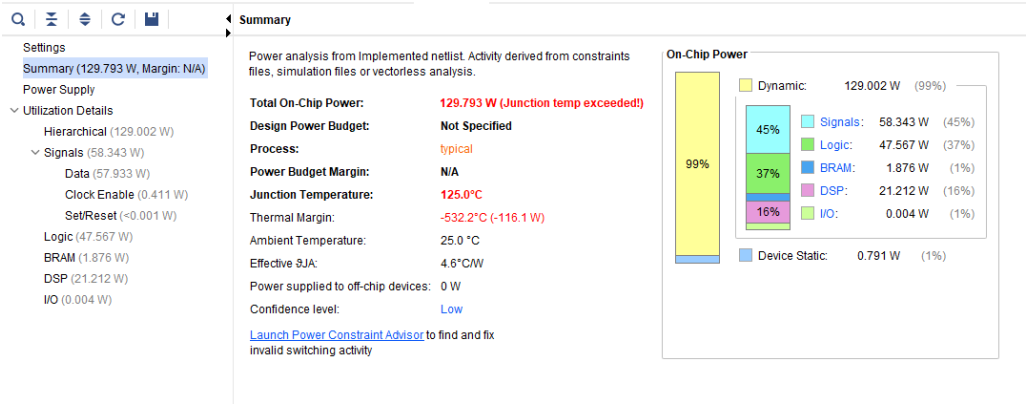


Figure 2: Power Report

3.3 FFT Plots Before and After Filtering

The spectrum of the raw audio signal and the filtered audio signal is compared in this subsection. The plots show how the filter modifies the spectral content of the input signal.

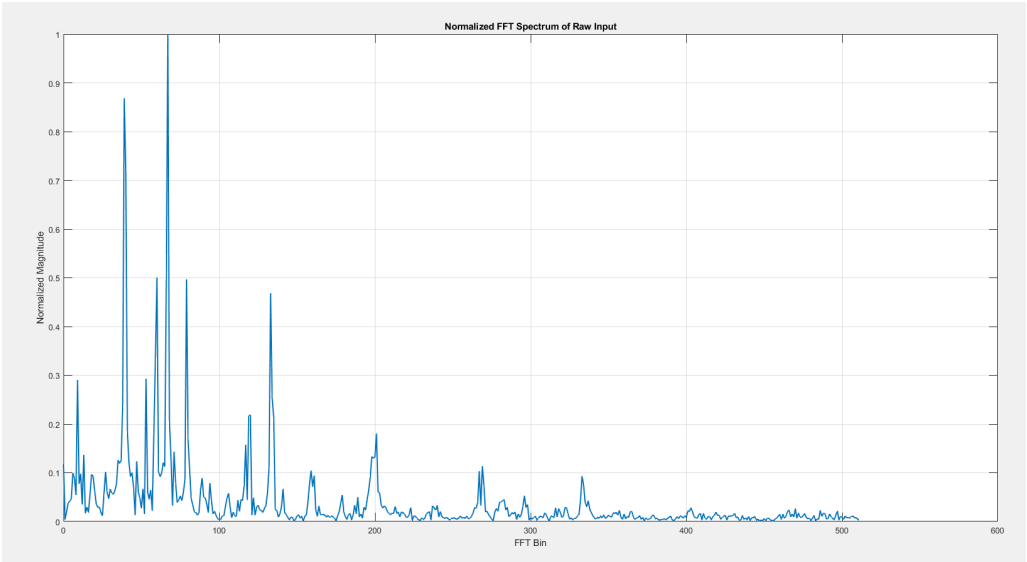


Figure 3: FFT magnitude plot of the raw audio signal

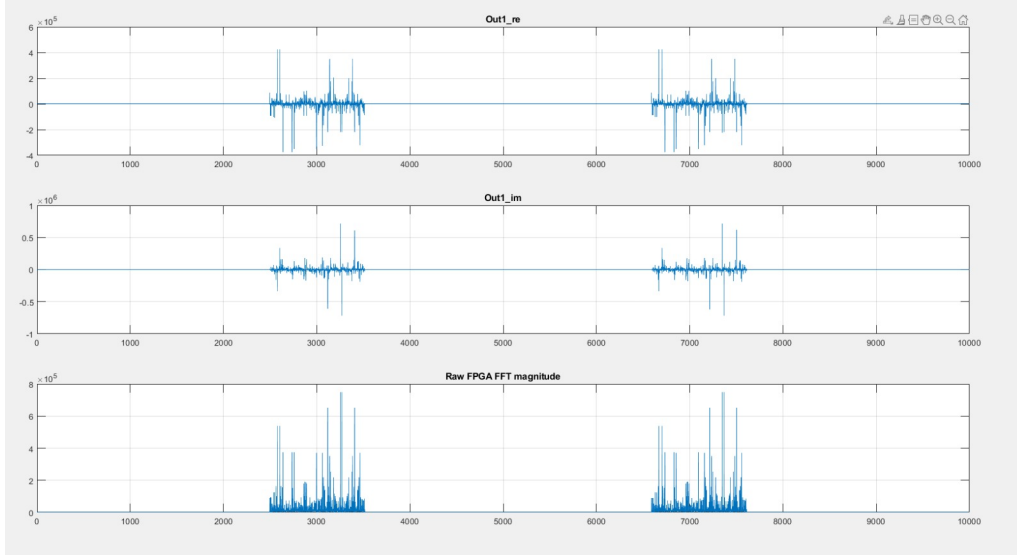


Figure 4: FFT magnitude plot of the filtered audio signal

The comparison between Figures 3 and 4 indicates the effect of filtering in suppressing or attenuating selected frequency components. The filtered spectrum reflects the characteristics of the implemented FIR filter.

3.4 Device Utilization

The device utilization report provides an estimate of the FPGA hardware resources consumed by the implemented design. This analysis is important for understanding the hardware cost of the proposed system and for evaluating whether the selected FPGA device is suitable for the required signal processing operations.

The main resources considered in this work include Look-Up Tables (LUTs), Flip-Flops (FFs), Block RAMs (BRAMs), and DSP slices. LUTs are primarily used for combinational logic, flip-flops are used for sequential storage elements, BRAMs are used for memory-related operations, and DSP slices are used for arithmetic operations such as multiplication and accumulation. Since the proposed design includes FFT and FIR filtering operations, the utilization of logic resources and memory resources is especially significant.

The utilization results obtained after synthesis and implementation are summarized below

3. Memory

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	9	0	0	135	6.67
RAMB36/FIFO*	2	0	0	135	1.48
RAMB36E1 only	2				
RAMB18	14	0	0	270	5.19
RAMB18E1 only	14				

Figure 5: Memory utilization report of the implemented FPGA design

4. DSP

Site Type	Used	Fixed	Prohibited	Available	Util%
DSPs	87	0	0	240	36.25
DSP48E1 only	87				

Figure 6: DSP utilization report of the implemented FPGA design

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	11846	0	0	63400	18.68
LUT as Logic	10201	0	0	63400	16.09
LUT as Memory	1645	0	0	19000	8.66
LUT as Distributed RAM	1124	0			
LUT as Shift Register	521	0			
Slice Registers	20039	0	0	126800	15.80
Register as Flip Flop	20039	0	0	126800	15.80
Register as Latch	0	0	0	126800	0.00
F7 Muxes	188	0	0	31700	0.59
F8 Muxes	0	0	0	15850	0.00

Figure 7: Overall device utilization summary of the FPGA implementation

The device utilization results indicate the extent to which the FPGA resources are occupied by the design. A higher utilization of LUTs and FFs is expected due to the logic-intensive implementation of FFT stages and control circuitry. Similarly, BRAM usage reflects memory requirements for storing coefficients, samples, or intermediate data, while DSP slice usage represents the arithmetic complexity of the filtering and transform operations.

These results help in assessing the scalability of the design. If the utilization is low to moderate, the design can be further extended with additional functionality such as real-time interfaces, higher-order filtering, or more advanced signal analysis blocks. On the other hand, very high utilization may require architectural optimization, resource sharing, or migration to a larger FPGA device.

4. Conclusion

This work demonstrated the implementation of an FPGA-based audio signal processing system involving FFT analysis and FIR filtering. The design methodology included sample preparation, hardware-oriented processing, HDL implementation, and post-implementation evaluation. The obtained FFT plots before and after filtering verify the expected spectral behavior of the system.

The implementation results show that FPGA platforms are suitable for real-time digital signal processing due to their parallel architecture and high throughput capability. Timing, power, and resource analysis further provide insight into the practical feasibility of the design.

5. Future Scope

The current design can be extended in several directions for improved functionality and performance:

- Real-time audio acquisition using external interfaces instead of memory-based input
- Direct transmission of processed data to a host system through UART or Ethernet
- Integration of higher-order or adaptive filters
- Real-time audio playback of filtered output
- Optimization of FFT architecture for reduced resource usage
- Development of a complete FPGA-based spectrum analyzer

6. References

1. Xilinx, "Vivado Design Suite User Guide: Power Analysis and Optimization" (UG907).
2. MathWorks, "HDL Coder Documentation: Generating HDL Code from Simulink".
3. Oppenheim, A. V., and Schafer, R. W., "Discrete-Time Signal Processing".

7. Individual Contribution

Table 3: Individual contribution of group members

S. No.	Name	Contribution
1	Ashwin Shankar	FIR Filter design and HDL code synthesis, FFT analysis
2	Nikita Nayak	Helped synthesize audio using MATLAB and created report
3	Shadan	Helped create FIR filter and acquired hardware
4	Rhea	Helped create report and acquired hardware
5	Aditya	Initial research of project constraints and audio synthesis